Guion del video · 5 minutos 45 segundos · 2 integrantes

Tiempo	Quién	Criterio de la rúbrica	Peso
0:00 – 2:45	**Integrante 1**	Arquitectura + Servicio desplegado + Infraestructura como Código	10 + 10 + 25 %
2:45 – 5:45	**Integrante 2** (dueño del repo)	Pipeline CI/CD + Desempeño y monitoreo	25 + 30 %

Reglas que cumple: cada uno abre **su propia** cuenta de AWS Academy al empezar y ejecuta comandos en vivo en ella. Los dos tienen la misma infraestructura desplegada, cada uno en su cuenta y con el mismo código.

Cómo grabar: hagan una reunión de Meet, Teams o Zoom, prendan la grabación y graben **una sola toma, sin cortes**. Cada uno comparte **su** pantalla cuando le toca. Todos los comandos se escriben en **Git Bash**, desde la carpeta del repositorio.

> El texto en *cursiva* es lo que se dice. No hace falta decirlo palabra por palabra; lo importante es mencionar las ideas en **negrita**.

INTEGRANTE 1 · 0:00 – 2:45

0:00 – 0:15 · Mi cuenta (15 s)

Pantalla: pestaña del Learner Lab, con el círculo **AWS** ● en verde. Clic en **AWS**, y en la consola señalar arriba a la derecha `voclabs/user…` y el número de cuenta.

"Soy [nombre]. Esta es mi cuenta de AWS Academy: el laboratorio está activo y esta es mi consola, con mi número de cuenta."

0:15 – 1:05 · Arquitectura (50 s)

Pantalla: el diagrama (`docs/arquitectura-aws.png` o el `.drawio` abierto en diagrams.net).

*"Desplegamos en us-east-1 una **VPC 10.0.0.0/16** en **dos zonas de disponibilidad**, para que si cae una zona el servicio siga. Cada zona tiene **tres subredes**: una pública con el balanceador y el NAT, una privada de aplicación con las EC2 y una privada de base de datos **sin ruta a Internet**.*

*Los usuarios entran por el Internet Gateway a un **Application Load Balancer**, que revisa las instancias con un **health check a /health** y reparte el tráfico. Las EC2 están en un **Auto Scaling Group de 2 a 4 instancias** que escala cuando la **CPU pasa del 50 %** y reemplaza solo cualquier instancia que falle: eso es la **alta disponibilidad**.*

*La seguridad son **tres Security Groups encadenados**: el balanceador recibe el puerto 80 desde Internet, las EC2 solo aceptan el 8080 desde el balanceador y la base de datos solo el 3306 desde las EC2. Ninguna EC2*

*tiene IP pública. Los datos están en **RDS MySQL con una réplica de lectura** en la otra zona, S3 guarda el código y los reportes del pipeline, y CloudWatch monitorea todo."*

Hacer (10 s): consola → buscar **VPC** → **Subnets** → escribir `dgiti-ef` en el filtro. *"Y aquí están las seis subredes del diagrama, creadas en mi cuenta."*

1:05 – 1:35 · Servicio desplegado (30 s)

1. En el navegador, abrir la URL del servicio (`app_url`) y **recargar 3 veces** (F5). *"Cada recarga la atiende **otra instancia en otra zona**: el balanceador está repartiendo."*

2. En Git Bash:

```
./scripts/demo_servicio.sh
```

*"**/health** es el endpoint de validación del balanceador. **/status** confirma la conexión con la base **primaria y con la réplica**. El **POST** escribe una visita en la primaria y el **GET** la lee desde la réplica: dice `read_from: replica` ."*

1:35 – 2:45 · Infraestructura como Código (70 s)

Pantalla: VS Code con la carpeta `terraform/` abierta a la izquierda y la terminal Git Bash abajo.

```
cd terraform
terraform init
terraform plan -var="app_version=1.1.0" -out=tfplan
```

Mientras el plan consulta AWS, hacer clic en cada archivo y decir:

*"**provider.tf** define el proveedor AWS y la región, **sin credenciales en el código**. **variables.tf** parametriza la red, el cómputo y la base de datos; la contraseña es **sensible** y tiene validación. **main.tf** crea todo: la red, los Security Groups, RDS con su réplica, S3, el balanceador, el Auto Scaling Group y el dashboard de CloudWatch. **user_data.sh** es lo que ejecuta cada EC2 al arrancar: descarga la app desde S3 y la levanta. **outputs.tf** entrega la URL y los endpoints. El **terraform.tfvars** con la contraseña **no se sube a Git**; solo el ejemplo. Nada está creado a mano."*

Cuando termine el plan: *"El plan detecta un solo cambio, la **versión de la aplicación**: Terraform crea una versión nueva del Launch Template y actualiza el Auto Scaling Group para que las instancias nuevas arranquen con esa versión; el pipeline es el que las reemplaza de una en una, sin cortar el servicio."*

```
terraform apply tfplan
terraform output
```

Al ver **Apply complete!**: *"Aplicado. Y estos son los outputs: la URL del servicio, los endpoints de la base de datos, el bucket y el dashboard. Ahora sigue el Integrante 2."*

INTEGRANTE 2 · 2:45 – 5:45

2:45 – 3:00 · Mi cuenta (15 s)

Pantalla: su Learner Lab con **AWS ●** en verde y su consola con su número de cuenta.

"Soy [nombre]. Esta es mi cuenta de AWS Academy, con mi laboratorio activo y mi propia infraestructura desplegada."

3:00 – 3:10 · Lanzar la prueba de carga (10 s)

En la **terminal 1** de Git Bash:

```
./scripts/prueba_carga.sh
```

*"Lanzo una prueba de carga con k6 contra mi balanceador: **dos minutos, hasta 100 usuarios virtuales**. Vuelvo a ella al final."*

3:10 – 4:10 · Pipeline CI/CD (60 s)

1. GitHub → archivo `.github/workflows/ci-cd.yml` abierto (10 s): *"El pipeline corre con cada push a main y tiene **tres etapas**. La uno valida el código con **flake8** y corre **15 pruebas unitarias**. La dos valida Terraform con **fmt, validate y test**. Si las dos pasan, la tres **despliega**: se autentica en mi cuenta con **secretos de GitHub**, publica el código en S3, **reemplaza las instancias del Auto Scaling Group de una en una** y corre **pruebas de integración contra el balanceador real**."*

2. Settings → **Secrets and variables** → **Actions** (5 s): *"Las credenciales están solo como secretos: se ven los nombres, nunca los valores."*

3. Actions → abrir la ejecución **en verde** que hicieron antes de grabar (35 s): - Etapa **1** → paso *Pruebas unitarias con cobertura* → señalar `15 passed`. - Etapa **2** → paso *terraform validate* → señalar `Success! The configuration is valid.` - Etapa **3** → paso *Desplegar* → señalar `estado=Successful`; luego *Pruebas de integración* → señalar `4 passed`.

*"Todo en verde: 15 pruebas unitarias, la infraestructura validada, el despliegue exitoso y 4 pruebas de integración, incluida una que verifica que respondan **al menos dos instancias distintas**."*

4. Consola → **S3** → bucket `dgiti-ef-artifacts-…` → carpeta `ci/` (10 s): *"Y aquí están los reportes de pruebas que guardó el pipeline."*

> ⚠ **No hagan push a main ni usen "Run workflow" durante la grabación**, porque el despliegue reemplazaría instancias en medio de la prueba de carga.

4:10 – 5:35 · Desempeño y monitoreo (85 s)

1. En la **terminal 2** (10 s):

```
./loadtest/check_balancer.sh "$(terraform -chdir=terraform output -raw app_url)" 10
```

*"Mientras corre la carga, las respuestas vienen de instancias en las **dos zonas**: el ALB reparte."*

2. Consola → **EC2** → **Target Groups** → **dgiti-ef-tg** → **pestaña Targets** (10 s): *"Todas las instancias están **healthy** en el target group."*

3. CloudWatch → **Dashboards** → **dgiti-ef-dashboard**, con el rango en **3h** (35 s): *"En **solicitudes por minuto** se ven los picos de la prueba previa y de la actual. El **tiempo de respuesta p95** sube cuando aumentan los usuarios: ese es el **punto de saturación**. En **códigos HTTP** casi todo es 2XX. **Solicitudes por instancia** muestra que el tráfico se divide entre los targets. La **CPU del ASG** pasó el objetivo del 50 %, y por eso las **instancias en servicio subieron de dos** a más. En RDS la CPU queda baja: **la base de datos no es el cuello de botella**."*

4. EC2 → **Auto Scaling Groups** → **dgiti-ef-asg** → **pestaña Activity** (10 s): *"Aquí está el evento: el Auto Scaling **lanzó instancias nuevas por la alarma de CPU**."*

5. Volver a la **terminal 1** con el resumen de k6 (20 s). Leer los valores reales: *"Resultado de k6: el tiempo de respuesta **p95 fue de [valor de http_req_duration p(95)]**, con **[valor de http_req_failed] de errores** y un máximo de **100 usuarios virtuales**. Los contadores instance_hits muestran que la zona A atendió [número] solicitudes y la zona B [número]: el balanceador repartió de forma pareja. Al saturar, la latencia subió por la CPU de las t3.micro; CloudWatch lo registró y el Auto Scaling agregó instancias."*

Si k6 marca ✗ en `http_req_duration`, agregar: *"Ese umbral de 1,5 segundos se superó en el pico de usuarios: es la evidencia de saturación que buscábamos."*

5:35 – 5:45 · Cierre (10 s)

"En resumen: infraestructura automatizada con Terraform, desplegada con CI/CD, validada bajo carga y monitoreada en CloudWatch. Gracias."

Si algo falla durante la grabación

Problema	Qué hacer
`ExpiredToken` o `InvalidClientTokenId`	Las credenciales vencieron: Learner Lab → **AWS Details** → **Show** → copiar de nuevo a `~/.aws/credentials` y repetir el comando
El plan muestra más cambios de lo esperado	Decir *"Terraform detecta diferencias con el código y las corrige"* y seguir con `terraform apply tfplan`
k6 todavía no termina	Abrir `loadtest/results/summary-larga.json` de la prueba previa, o la terminal 3 donde quedó, y leer esos valores
El ASG no escaló en vivo	Con el dashboard en **3h** se ve el escalado de la prueba previa, y en **Activity** aparecen sus eventos
`check_balancer.sh` muestra alguna línea ERROR	*"Es una instancia que estaba saliendo de servicio; el ALB deja de enviarle tráfico"*
Un target aparece `unhealthy` o `initial`	Es una instancia nueva que está arrancando (unos 3 minutos); el ALB todavía no le envía tráfico

Justificaciones (por si el docente pregunta)

Decisión	Por qué
2 zonas de disponibilidad	Si cae una zona, la otra sigue atendiendo
ALB (capa 7)	Revisa la aplicación con un health check HTTP y reparte peticiones HTTP
EC2 en subred privada	No se pueden alcanzar desde Internet; solo entra lo que pasa por el ALB
BD sin ruta a Internet	La base de datos no tiene camino hacia ni desde Internet
SG que referencian a otros SG	Cada instancia nueva del ASG queda autorizada automáticamente, sin depender de IPs
Un solo NAT Gateway	Ahorro de costo en el lab; solo afecta la salida de las EC2. En producción, uno por zona
Réplica de lectura	Separa lecturas y escrituras y deja una copia de los datos en otra zona
Sin Multi-AZ en RDS	Duplica el costo en el lab; en producción se activaría para la conmutación automática
t3.micro + escalado horizontal	Varias instancias pequeñas son más baratas y no son un punto único de falla
Escalado con CPU al 50 %	Deja margen para absorber picos mientras arrancan instancias nuevas
IMDSv2 y RDS cifrado	Buenas prácticas de seguridad sin costo adicional
Código en S3 con VPC endpoint	Las EC2 privadas descargan la app sin pasar por Internet
Reemplazo gradual en el pipeline	El despliegue cambia las instancias de a una y espera que la nueva esté sana; AWS Academy bloquea el *instance refresh* automático, por eso lo hace un script
Estado de Terraform local	Cada integrante despliega en su propia cuenta; en producción iría en un backend S3